SOLID Principles Implementation

Calculator Console Application — Assignment 3

Prepared by: Ashutosh Kumar Tiwari

Purpose: This document explains where each SOLID principle is used in the Calculator project and how the design improves maintainability, extensibility, and testability.

1. SOLID Principles Summary

Principle	Meaning	Used In Project	Benefit
SRP	A class should have only one responsibility.	InputManager, DisplayManager, FileManager, HistoryManager, Calculator	Code is easier to understand and change.
OCP	Classes should be open for extension but closed for modification.	ICalculatorOperation and separate operation classes	New operations can be added with minimal changes.
LSP	Child classes should behave like the parent abstraction.	AdditionOperation, SubtractionOperation, MultiplicationOperation, DivisionOperation	Any operation object can replace ICalculatorOperation safely.
ISP	Interfaces should be small and focused.	IDisplayManager, IInputManager, IFileManager, IOperationFactory	Classes do not depend on unused methods.
DIP	High-level classes should depend on abstractions, not concrete classes.	Calculator receives interfaces through constructor injection	Loose coupling and easier testing.

2. SRP — Single Responsibility Principle

The Calculator project follows SRP by separating responsibilities into different classes. Each class has one main reason to change.

- InputManager handles user input and validation.
- DisplayManager handles console output and messages.
- FileManager handles saving and reading calculation history from file.
- HistoryManager manages calculation history data.
- Calculator controls the application workflow.

3. OCP — Open/Closed Principle

The project follows OCP by creating separate classes for each calculator operation. Existing operation classes do not need to be modified when a new operation is added.

- AdditionOperation performs addition.
- SubtractionOperation performs subtraction.
- MultiplicationOperation performs multiplication.
- DivisionOperation performs division.
- To add PowerOperation, create a new class instead of changing existing operation classes.

4. LSP — Liskov Substitution Principle

The project follows LSP because every operation class can be used wherever the common operation abstraction is expected.

- ICalculatorOperation operation = new AdditionOperation(); works.
- ICalculatorOperation operation = new DivisionOperation(); also works.
- The Calculator can call Execute() without knowing the exact child class.
- No child operation should break the expected behavior of Execute().

5. ISP — Interface Segregation Principle

The project follows ISP by using small interfaces instead of one large interface. Each class implements only the methods it actually needs.

- IDisplayManager contains display-related methods only.
- IInputManager contains input-related methods only.
- IFileManager contains file-related methods only.
- IOperationFactory contains operation creation responsibility only.

6. DIP — Dependency Inversion Principle

The project follows DIP by making the Calculator class depend on interfaces instead of concrete classes.

- Calculator depends on IInputManager, IDisplayManager, IFileManager, IHistoryManager, and IOperationFactory.
- Objects are injected through the constructor.
- Calculator does not directly create concrete classes like new DisplayManager() inside business logic.
- This makes the code loosely coupled and easier to test.